# SOMA

## *Self-Organising Modular Architecture*
## *Comprehensive Engineering Specification*

**Organisation:** Wakasa Labs · Nairobi, Kenya

**Version:** 0.1 · Engineering Draft

**Document type:** Internal Engineering Reference

**Date:** March 2026

**Target:** Paper 1 — Selective Growth for Continual Learning

**Status:** Active Development

**PURPOSE OF THIS DOCUMENT**

This document is the single source of truth for the SOMA engineering effort. It covers everything from the mathematical problem definition through to exact terminal commands for running experiments. Nothing is assumed. Read it once and know everything needed to build, test, and validate SOMA.

SECTION 1

# Executive Summary

SOMA (Self-Organising Modular Architecture) is a continual learning framework for large language models that solves catastrophic forgetting by growing new LoRA adapter capacity when — and only when — existing capacity is genuinely exhausted. It does not protect old weights through regularisation penalties or experience replay. It makes forgetting structurally impossible by freezing every adapter immediately after training.

The core innovation is a three-signal necessity detector (SOMA-NECESSITY) that must unanimously agree before any new capacity is created. This prevents the primary failure mode of prior methods: over-spawning on false positives (e.g. learning rate decay plateau, not true capacity exhaustion). An RL controller (SOMA-GROW) then decides HOW to grow — spawn new, update existing with gating, merge, or skip — and learns from the reward signal over time.

| Metric | Target | Baseline comparison |
|---|---|---|
| *Backward Transfer (BT)* | **> -0.05** | EWC: -0.09 \| GEM: -0.08 \| Replay: -0.07 \| No-protection: -0.18 |
| *Final adapter count K* | **< 15** | Always-spawn: K=n_tasks \| SOMA target: K < n_tasks * 0.6 |
| *N3 fire rate (systematic tasks)* | **> 70%** | Should fire on most tasks in permuted benchmark (all systematic) |
| *N3 fire rate (handled tasks)* | **< 15%** | Should NOT fire when existing adapter already handles task well |
| *Build cost to Paper 1* | **< $133** | Kaggle free T4 (30hr/week) + Google Colab credit |

SECTION 2

# The Problem — Formal Definition of Catastrophic Forgetting

Before defining what SOMA solves, we need a precise statement of the problem. Every design decision in SOMA traces back to this definition.

| Def. 2.1 — Sequential Task Learning | |
|---|---|
| `Given tasks T_1, T_2, ..., T_n arriving one at a time, train a model M on each task using only data from that task.` | *The model sees each task once, in sequence. No joint training. No stored data from past tasks.* |

| Def. 2.2 — Catastrophic Forgetting | |
|---|---|
| `A(M_t, T_i) << A(M_i, T_i) for i < t where M_t is the model after training on T_t.` | *After learning task t, accuracy on earlier task i collapses. The model has 'forgotten'.* |

| Def. 2.3 — Backward Transfer (BT) | |
|---|---|
| `BT = (1/T-1) * sum_{i=1}^{T-1} [A(M_T, T_i) - A(M_i, T_i)]` | *Average accuracy change on old tasks. BT = 0 means perfect retention. BT = -0.18 means severe forgetting.* |

| Def. 2.4 — SOMA Target | |
|---|---|
| `Achieve BT > -0.05 on sequential benchmarks using K << n_tasks adapters (efficient growth).` | *Field target: beat EWC (-0.09) and replay (-0.07) simultaneously. K must stay bounded.* |

## Why Standard Fine-tuning Fails

A neural network with weight vector theta in R^n encodes all knowledge in those n numbers. When you run gradient descent on a new task, you move theta to minimise the new task's loss. But the gradient has no memory of the old tasks. It pushes theta wherever the new task needs it to go, overwriting the configurations that encoded old knowledge. This is not a bug — it is the exact behaviour gradient descent is designed to produce.

The only escape is structural: prevent the overwriting by construction. SOMA does this by freezing the base model weights permanently and training only LoRA adapters — low-rank weight perturbations — one per task cluster. Frozen adapters cannot be overwritten. Forgetting becomes architecturally impossible.

**KEY CONCEPT: LoRA Adapters**

LoRA (Low-Rank Adaptation): delta_W = B * A where B is [d x r] and A is [r x d], r << d. Instead of updating all d*d weights, only 2*d*r parameters are trained. With r=8 and d=768 (BERT-base), that is 12,288 parameters vs 589,824 — a 48x reduction. After training, B and A are frozen. The base model is never touched.

SECTION 3

# What We Have Solved

Eight open problems were identified in the original SOMA specification. Two are solved in this document and are required for Paper 1. The remainder are deferred to Papers 2 and 3.

| # | Problem | Required for | Status |
|---|---|---|---|
| 1 | Capacity Trigger — when is a new adapter genuinely needed? | *Paper 1* | SOLVED |
| 2 | Merge Operation — combine two LoRAs without knowledge loss | *Paper 1 (partial)* | PARTIAL |
| 3 | Router Forgetting — router itself forgets old assignments | *Paper 1* | PARTIAL |
| 4 | Growth Ceiling — when to stop growing and force consolidation | *Paper 1* | SOLVED |
| 5 | Fair Evaluation — accuracy metric normalised per parameter | *Paper 1* | SOLVED |
| 6 | Cold Start — bootstrap for first 2-3 tasks | *Paper 1* | SOLVED |
| 7 | Systematic Failure — unsupervised detection of error categories | *Paper 1* | SOLVED |
| 8 | Mutation Mapping — failure type to architectural change | *Paper 3* | DEFERRED |

## 3.1 Problem 1 — Capacity Trigger (SOLVED)

**DESIGN CHANGE FROM PREVIOUS SESSION**

Previous approach: N2 used cosine similarity between new and old gradients. This was changed after reviewing KeepLoRA (Jan 2026) and InfLoRA (CVPR 2024).

The core question of Problem 1 is: **when is a model failing because it needs more capacity versus because it needs more training?** These two causes look identical from the outside — loss is high — but require opposite responses.

The solution uses three jointly-necessary conditions. Each condition rules out one false positive class:

| Condition | What it measures | False positive it prevents | Formula |
|---|---|---|---|
| `N1 — Loss Plateau` | Training has genuinely stalled (not just a bad batch) | *Triggering growth on noisy loss spikes* | `\|L(t) - L(t-p)\| < epsilon_loss over p=300 steps` |
| `N2 — Subspace Saturation` | New gradient is orthogonal to existing principal subspace | *Triggering growth on tasks already covered by existing adapters* | `residual = \|\|g - P(P^T g)\|\|^2 / \|\|g\|\|^2 > 0.80` |
| `N3 — Systematic Failure` | Error cases cluster consistently (not random noise) | *Triggering growth on stochastic noise that will resolve with more training* | `DBSCAN clusters failure gradients: silhouette > 0.30 AND entropy < calibrated_threshold` |

## 3.2 Problem 7 — Systematic Failure Detection (SOLVED)

The key insight is that we should cluster **failure-case gradients**, not hidden states. Gradients encode "what weight change would fix this error". If all failures need the same weight change — their gradients point in the same direction — the failures share a single structural cause. That is a systematic gap.

Random failures need different fixes. Their gradients point in random directions. No clustering emerges. The system correctly identifies this as noise and does not grow.

**DESIGN RATIONALE: WHY GRADIENTS NOT HIDDEN STATES**

Why gradient clustering over hidden-state clustering: (1) Gradients are already computed for N2 — zero extra forward passes. (2) Hidden states encode 'where the model is uncertain' — we care about 'what would fix it'. (3) TreeLoRA (ICML 2025) validates gradient similarity as the correct space for adapter organisation — we extend this to failure detection specifically.

SECTION 4

# System Architecture

SOMA has five interacting components. Understanding their data flow is necessary before reading any implementation code.


*Data flow: Task Stream feeds SOMA-NECESSITY. NECESSITY signal gates SOMA-GROW. GROW updates the Adapter Pool. Router is updated whenever Pool changes.*

## 4.1 Component Responsibilities

### SomaNecessity

`core/necessity.py`

Runs continuously during training on a new task. Monitors loss trajectory (N1), measures how much of the new gradient falls outside the learned subspace (N2), and clusters failure-case gradients to detect systematic gaps (N3). Returns a NecessityResult dataclass with boolean signals and continuous scores.

**Pass criterion: All three signals TRUE simultaneously**

### SomaGrow

`core/grow.py`

Receives the necessity signal and current system state, then selects one of four actions: UPDATE_EXISTING (retrain existing adapter with KL gating), SPAWN_NEW (create fresh adapter), MERGE (combine two most-similar adapters when K >= max_K), or SKIP. Computes reward and updates the RL policy via REINFORCE.

**Pass criterion: Correct action selected; reward is positive**

### GrowthPolicy

`core/grow.py`

Linear policy (W: [7,4]) trained by REINFORCE. Input: 7-dimensional state vector. Output: probability distribution over 4 actions. Paper 1 uses linear policy for simplicity. Paper 2 will upgrade to PPO via stable-baselines3.

**Pass criterion: Policy reward increases over first 10 tasks**

### SomaRouter

`core/router.py`

Prototype-based nearest-neighbour router. Stores n_prototypes embeddings per adapter from that adapter's training task. Routes new inputs by cosine similarity to stored prototypes. Does not use gradient updates, so it cannot suffer catastrophic forgetting.

**Pass criterion: Routing accuracy > 80% on previously-seen tasks**

### SomaLearn

`core/learn.py`

The outer training loop. Orchestrates all components across the task stream. Handles cold start (first 2 tasks always spawn). Enforces growth ceiling. Computes BT and FT metrics at each eval interval. Writes JSON logs.

**Pass criterion: BT > -0.05 after 10 tasks**

SECTION 5

# Algorithm Specifications

Complete pseudocode for all three SOMA algorithms. These are the reference specifications — implementation must match exactly.

## 5.1 SOMA-NECESSITY

**SOMA-NECESSITY Part 1: N1 — Loss Plateau** *python*

```
Algorithm SOMA-NECESSITY(task T_t, adapter Phi_i, config cfg)
Returns: NecessityResult{necessity, n1, n2, n3, scores}

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
N1: LOSS PLATEAU
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Train Phi_i on T_t for p=300 gradient steps, recording loss at each step.
Compute smoothed_loss = mean(loss_history[-window:]) where window=30
If best_smoothed - smoothed_loss < epsilon_loss=0.001 for p consecutive steps:
N1 = TRUE # training is genuinely stuck
Else:
N1 = FALSE # still improving, give it more time
plateau_score (continuous) = min(1.0, stale_steps / plateau_patience)
```

**SOMA-NECESSITY Part 2: N2 — Subspace Saturation** *python*

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
N2: SUBSPACE SATURATION (revised — KeepLoRA 2026 method)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
P = existing_principal_subspace_basis # shape [d, rank=16]
P is accumulated after each completed task via SVD:
G_completed = stack(task_gradients) # [n, d]
_, _, Vt = svd(G_completed) # thin SVD
new_vecs = Vt[:rank].T # [d, rank]
P = QR_orthogonalise(concat(P, new_vecs))[:,:rank]

g_new = mean(new_task_gradients) # [d]
g_new = g_new / norm(g_new)
proj = P @ (P.T @ g_new) # projection onto P
residual = g_new - proj
residual_fraction = dot(residual, residual) / dot(g_new, g_new)

N2 = TRUE iff residual_fraction > 0.80
# High residual = new task is orthogonal to P = existing adapters can't help
```

**SOMA-NECESSITY Part 3: N3 + Conjunction** *python*

```
═══════════════════════════════════════════════════════════════
N3: SYSTEMATIC FAILURE DETECTION
═══════════════════════════════════════════════════════════════
Collect {g_i : gradient for each failed prediction during training}
If len(failure_grads) < min_failures=25: N3 = FALSE (insufficient data)

G_fail = stack(failure_grads) # [n_fail, d]
G_fail -= mean(G_fail, axis=0) # centre
_, _, Vt = svd(G_fail) # thin SVD
G_proj = G_fail @ Vt[:proj_dim=32].T # [n_fail, 32]
G_norm = G_proj / norm(G_proj, axis=1, keepdims=True) # cosine space

labels = DBSCAN(eps=0.5, min_samples=5, metric='cosine').fit(G_norm)
n_clusters = count_non_noise_clusters(labels)
If n_clusters < 2: N3 = FALSE # no cluster structure

sil = silhouette_score(G_norm, labels, metric='cosine') # cluster quality
p_k = bincount(labels) / n_labelled # cluster sizes
entropy = -sum(p_k * log(p_k)) # concentration

# Entropy threshold is calibrated from cold-start tasks (not fixed):
# threshold = 0.70 * mean(baseline_entropy_task1, baseline_entropy_task2)

N3 = TRUE iff (sil > 0.30) AND (entropy < calibrated_threshold)

═══════════════════════════════════════════════════════════════
CONJUNCTION
═══════════════════════════════════════════════════════════════
NECESSITY = N1 AND N2 AND N3
Return NecessityResult(necessity, n1, n2, n3, plateau_score,
residual_fraction, sil, entropy)
```

## 5.2 SOMA-GROW

**SOMA-GROW Part 1: State + Action Selection** *python*

```
Algorithm SOMA-GROW(state s_t, necessity, policy pi_1, pool, data)
Returns: GrowResult{action, reward, k_before, k_after}

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
STATE VECTOR (7 features, all normalised to [0,1]):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
s[0] = plateau_score # N1 continuous: 0=training fine, 1=fully plateaued
s[1] = residual_fraction # N2 continuous: 0=in subspace, 1=fully orthogonal
s[2] = failure_entropy # N3 continuous: low=systematic, high=random
s[3] = n_failures / 25 # normalised failure count
s[4] = K / max_K # adapter pool fullness
s[5] = router_max_confidence # how sure router is about routing
s[6] = steps_since_spawn / 20 # recency of last spawn (prevents oscillation)

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ACTION SELECTION:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
If K >= max_K: force action = MERGE (growth ceiling enforcement)
Else: action = pi_1.select(s_t) # policy softmax over 4 actions

Actions: 0=UPDATE_EXISTING, 1=SPAWN_NEW, 2=MERGE, 3=SKIP
```

**SOMA-GROW Part 2: Execution + Reward** *python*

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
ACTION EXECUTION:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
If action == UPDATE_EXISTING:
i = best_matching_adapter(pool, state)
B_old, A_old = pool[i]
B_new, A_new = train(pool[i], task_data, lr=lr_default)
kl = ||B_new*A_new - B_old*A_old||_F / ||B_old*A_old||_F
If kl < kl_gate=0.10:
pool[i] = (B_new, A_new) # accept update
Else: # gating rejected, rescale
B_new2, A_new2 = train(pool[i], task_data, lr=lr*0.5)
kl2 = kl_proxy(B_new2, A_new2, B_old, A_old)
If kl2 < kl_gate: pool[i] = (B_new2, A_new2)
Else: reject entirely (log rejection)

If action == SPAWN_NEW:
B_new, A_new = train_fresh(task_data, rank=8)
pool.append((B_new, A_new))
router.register(len(pool)-1, embed(task_data))

If action == MERGE:
i, j = most_similar_pair(pool) # cosine similarity of B*A
pool[i] = average_merge(pool[i], pool[j]) # [Problem 2: partial]
pool.remove(j); router.remove(j)

If action == SKIP: pass

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
REWARD:
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
delta_acc_new = acc(pool, task_t) - acc_before
delta_BT = mean(acc(pool, task_i) - acc_before_i for i in past)
delta_K = K_after - K_before
reward = alpha=1.0 * delta_acc_new
- beta=2.0 * abs(delta_BT) # forgetting penalised harder
- gamma=0.5 * max(0, delta_K) # growth penalised lightly

pi_1.record(s_t, action, reward) # buffer for REINFORCE update
```

## 5.3 SOMA-LEARN (Outer Loop)

**SOMA-LEARN: Complete Outer Loop** *python*

```
Algorithm SOMA-LEARN(task_stream, base_model theta, max_K=20)
Returns: trained SOMA system

═══════════════════════════════════════════════════════════════
INITIALISATION:
═══════════════════════════════════════════════════════════════
K = 0; pool = []; history = []
necessity = SomaNecessity(cfg) # N1+N2+N3 detector
grow = SomaGrow(cfg) # RL policy + actions
router = SomaRouter(cfg) # prototype router

═══════════════════════════════════════════════════════════════
COLD START (tasks 0 and 1: no routing history, always spawn):
═══════════════════════════════════════════════════════════════
For t in [0, 1]:
B, A = train_fresh(T_t, theta) # spawn unconditionally
pool.append((B, A))
router.register(K, embed(T_t))
necessity.task_completed(grads_this_task) # calibrate N3 threshold
K += 1

═══════════════════════════════════════════════════════════════
MAIN LOOP (t >= 2):
═══════════════════════════════════════════════════════════════
For each task T_t in stream:
necessity.reset_for_task()

# Run training on T_t, feeding signals to necessity detector:
For each batch in T_t:
loss = model(batch)
grad = gradient(loss, adapter_params)
necessity.update_loss(loss)
necessity.add_gradient(grad)
If model_was_wrong: necessity.add_failure_gradient(grad)

nec_result = necessity.check()
state = build_state_vector(nec_result, router, K, max_K)

If K >= max_K: force_action = MERGE
Else: force_action = None

grow_result = grow.step(state, nec_result, pool, T_t,
past_tasks, train_fn, eval_fn, force_action)
necessity.task_completed(grads_this_task)

If t % 5 == 0: grow.update_policy() # REINFORCE update

Log: task_idx, necessity, action, reward, K, BT, FT

Return (theta, pool, router, grow.policy)
```

SECTION 6

# Implementation Reference

Detailed documentation for every class and method. Read this before touching the code.

## 6.1 necessity.py — NecessityConfig

| Parameter | Type | Default | Description |
|---|---|---|---|
| `plateau_patience` | *int* | `300` | Steps without improvement before N1=True. Lower = more trigger-happy. Start at 300, tune down if over-spawning. |
| `plateau_min_delta` | *float* | `1e-3` | Minimum meaningful loss improvement. Below this counts as no improvement. |
| `plateau_window` | *int* | `30` | Rolling window size for loss smoothing. Higher = smoother, slower to react. |
| `subspace_rank` | *int* | `16` | Number of principal SVD directions to keep. Higher = richer subspace, more memory. |
| `residual_threshold` | *float* | `0.80` | Fraction of gradient energy in residual subspace to declare saturation. 0.80 from SplitLoRA analysis. |
| `min_failures` | *int* | `25` | Minimum failure cases needed before N3 clustering. Below this, returns False. |
| `proj_dim` | *int* | `32` | Gradient projection dimension for DBSCAN. 32 is fast and sufficient for clustering. |
| `silhouette_min` | *float* | `0.30` | Minimum silhouette score for clusters to be considered real. Below 0.30 = noise. |
| `entropy_default` | *float* | `0.70` | Fallback entropy threshold used before calibration runs. Tune after calibration. |
| `calibration_tasks` | *int* | `2` | Number of cold-start tasks used to calibrate the N3 entropy threshold. |

## 6.2 necessity.py — Key Methods

| Method signature | Description |
|---|---|
| `update_loss(loss: float) -> bool` | Call every training step. Updates N1 plateau detector. Returns True if plateau detected. |
| `add_gradient(g: np.ndarray)` | Call every training step. Buffers gradient for N2 subspace check. Shape must be flat [d]. |
| `add_failure_gradient(g: np.ndarray)` | Call when model prediction is wrong. Buffers gradient for N3 clustering. Same shape as add_gradient. |
| `task_completed(grads: list[np.ndarray])` | Call after finishing training on a task. Updates principal subspace basis. Handles N3 calibration for first 2 tasks. |

| | |
|---|---|
| `reset_for_task()` | Call before starting a new task. Clears N1 history, N2 buffer, N3 failure buffer. Does NOT clear subspace basis. |
| `check() -> NecessityResult` | Runs full N1+N2+N3 check. Call after sufficient training. Returns NecessityResult with all signals and continuous scores. |
| `rl_state_features() -> np.ndarray` | Returns 4 continuous features for RL state vector: [plateau_score, residual_fraction, failure_entropy, failures_norm]. |

## 6.3 grow.py — GrowConfig

| Parameter | Type | Default | Description |
|---|---|---|---|
| `alpha` | *float* | `1.0` | Reward weight for new-task accuracy gain. Increase to encourage learning. |
| `beta` | *float* | `2.0` | Penalty weight for backward transfer (forgetting). beta > alpha = conservative. Do not lower below 1.5. |
| `gamma` | *float* | `0.5` | Penalty weight for adapter count increase. Keeps K from growing unnecessarily. |
| `kl_gate` | *float* | `0.10` | Maximum KL proxy for UPDATE gating. If update moves adapter more than 10% of its norm, rescale. |
| `max_k` | *int* | `20` | Growth ceiling. When K reaches max_k, next action is forced to MERGE regardless of policy. |

## 6.4 grow.py — GrowthPolicy (RL Policy)

The policy is a linear model: logits = W @ state + b, where W is [7 x 4] and b is [4]. Action probabilities are computed via softmax. Policy is trained by REINFORCE (policy gradient) on a buffered trajectory after every 5 tasks.

**GrowthPolicy — Training Notes** *python*

```
# State vector (7 features) -> action probabilities (4 actions)
# W shape: [7, 4] b shape: [4]
#
# Initialisation: W ~ Normal(0, 0.01), b = zeros
# This near-zero init means all actions start equally likely.
#
# REINFORCE update (every 5 tasks):
# 1. Compute discounted returns G_t for buffered trajectory
# 2. Normalise returns: G_t = (G_t - mean) / std
# 3. For each (s_t, a_t, G_t):
# grad_logits = softmax(s_t @ W + b)
# grad_logits[a_t] -= 1.0
# grad_logits *= G_t * lr
# W -= outer(s_t, grad_logits)
# b -= grad_logits
#
# Important: buffer is cleared after each update.
# lr=0.01 is conservative — policy changes slowly, which is correct.
```

## 6.5 router.py — SomaRouter

The router uses prototype-based nearest-neighbour matching. For each adapter, it stores up to n_prototypes=10 embedding vectors from that adapter's training examples. Routing uses mean cosine similarity to the prototype set. No gradient updates means no forgetting.

**DESIGN DECISION: PROTOTYPE VS CLASSIFIER ROUTING**

Why prototype routing instead of a trained classifier? A trained classifier would suffer catastrophic forgetting when new adapters are added (its output head would need to expand). Prototypes don't. Adding a new adapter just adds a new entry to the prototype dictionary — existing entries are untouched.

## 6.6 learn.py — Training Loop Integration

**Integration Guide — learn.py Usage** *python*

```
# HOW TO INTEGRATE SOMA INTO YOUR TRAINING LOOP
# (assuming you have a model and dataloader)

from soma.core.necessity import SomaNecessity, NecessityConfig
from soma.core.grow import SomaGrow, GrowConfig
from soma.core.router import SomaRouter
from soma.core.learn import SomaLearn, LearnConfig

# Step 1: Define injected functions
def train_fn(adapter_idx, task_data, lr_scale=1.0):
# adapter_idx = None -> spawn new adapter
# adapter_idx = int -> fine-tune existing adapter at that index
# Returns (B, A) numpy arrays
...

def eval_fn(adapter_idx, task_data):
# adapter_idx = -1 -> use best available adapter
# Returns accuracy as float in [0, 1]
...

def embed_fn(task_data):
# Returns [n, embed_dim] embeddings for routing
...

# Step 2: Instantiate SOMA
learn = SomaLearn(train_fn, eval_fn, embed_fn)

# Step 3: Run task stream
past_tasks = []
for task in task_stream:
log = learn.step(task_idx=t, task_data=task, past_task_data=past_tasks)
past_tasks.append(task)
print(f'BT: {log.backward_transfer:.4f} K: {log.k_after}')

learn.summary() # prints final metrics
```

SECTION 7

# Repository Structure

**Repository Structure — Part A**

```
soma/ ROOT PACKAGE
__init__.py package marker
requirements.txt all dependencies with versions

core/ ALGORITHM IMPLEMENTATION
__init__.py exports: SomaNecessity, SomaGrow, SomaLearn
necessity.py N1+N2+N3 detectors (Problems 1 & 7 solved)
grow.py RL policy, 4 actions, reward, KL gating
router.py prototype-based adapter routing
learn.py outer training loop, metrics, logging

experiments/ RUNNABLE EXPERIMENTS
__init__.py
run_permuted_mnist.py Paper 1 Experiment 1 (build first)
run_gsm8k_sequential.py Paper 1 Experiment 2 (build in Stage 5)
baselines/
__init__.py
ewc.py EWC baseline (Kirkpatrick et al.)
replay.py Experience replay baseline
sequential.py No-protection sequential fine-tuning
```

**Repository Structure — Part B**

```
configs/ HYPERPARAMETER CONFIGS
default.json base config (inherit from this)
paper1_permuted_mnist.json Experiment 1 config
paper1_gsm8k.json Experiment 2 config

utils/ SHARED UTILITIES
__init__.py
metrics.py BT, FT, per-param-normalised accuracy
logging.py wandb integration + JSON logging
checkpoint.py save/load adapter pool + policy weights
visualise.py BT/K curves, cluster visualisation

tests/ UNIT TESTS
__init__.py
test_n1_plateau.py N1: verify fires on plateau not on noise
test_n2_subspace.py N2: verify fires on orthogonal tasks
test_n3_clustering.py N3: verify fires on systematic not random
test_grow_reward.py reward function, KL gating
test_router.py routing accuracy, no forgetting
test_learn_integration.py full 3-task integration smoke test

notebooks/ KAGGLE/COLAB NOTEBOOKS
01_experiment1_permuted_mnist.ipynb ready to run on Kaggle T4
02_experiment2_gsm8k.ipynb ready to run on Kaggle T4
03_ablation_necessity_signals.ipynb ablation: remove each Ni separately

paper/ LATEX SOURCE
main.tex | figures/ | tables/

docs/ REFERENCE DOCUMENTS
SOMA_Mathematical_Foundations.pdf
SOMA_Algorithms.pdf
SOMA_Engineering_Specification.pdf (this document)
```

SECTION 8

# Environment Setup

Exact commands. Run these in order. Nothing assumed.

## 8.1 Kaggle Setup (Primary — Free T4 GPU)

**Kaggle Setup Commands** *python*

```python
# 1. Go to https://kaggle.com -> 'New Notebook'
# 2. Settings -> Accelerator -> GPU T4 x1
# 3. Settings -> Internet -> ON (needed for first pip install)

# In first cell — install dependencies:
!pip install peft==0.10.0 scikit-learn==1.4.0 wandb tqdm -q

# Clone or upload the SOMA package:
# Option A: upload soma/ folder as a dataset, then:
import sys
sys.path.insert(0, '/kaggle/input/soma-package')

# Option B: if using GitHub:
!git clone https://github.com/wakasa-labs/soma.git /kaggle/working/soma
sys.path.insert(0, '/kaggle/working')

# Verify installation:
from soma.core.necessity import SomaNecessity
from soma.core.grow import SomaGrow
from soma.core.learn import SomaLearn
print('SOMA loaded successfully')
```

## 8.2 Google Colab Setup (Secondary)

**Google Colab Setup Commands** *python*

```python
# Runtime -> Change runtime type -> GPU (T4 or A100)

!pip install torch==2.2.0 transformers==4.40.0 peft==0.10.0 \
scikit-learn==1.4.0 torchvision wandb tqdm -q

# Mount Drive if checkpointing:
from google.colab import drive
drive.mount('/content/drive')

# Clone repo:
!git clone https://github.com/wakasa-labs/soma.git
%cd soma

# Verify:
!python -c 'from soma.core.learn import SomaLearn; print("OK")'
```

## 8.3 Local Setup (Development/Testing)

**Local Development Setup** *python*

```
# Python 3.10+ required. 3.11 recommended.

# Create environment:
python -m venv soma_env
source soma_env/bin/activate # Linux/Mac
soma_env\Scripts\activate # Windows

# Install dependencies:
pip install -r soma/requirements.txt

# Install soma as editable package (lets you edit code and test immediately):
pip install -e .

# Verify:
python -m pytest soma/tests/ -v

# Expected output: all tests pass except test_n3 (needs 25+ failures)
# That's OK — see Section 10 for full test plan.
```

## 8.4 Verify GPU Availability

**GPU Verification** *python*

```
import torch
print(f'PyTorch version: {torch.__version__}')
print(f'CUDA available: {torch.cuda.is_available()}')
if torch.cuda.is_available():
print(f'GPU: {torch.cuda.get_device_name(0)}')
print(f'VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')

# Expected on Kaggle T4:
# CUDA available: True
# GPU: Tesla T4
# VRAM: 15.8 GB

# If CUDA not available, experiments will still run on CPU
# but Permuted MNIST will take ~45 min instead of ~15 min.
```

SECTION 9

# How to Run Each Experiment

Step-by-step instructions for every experiment. Copy-paste these commands.

## 9.1 Experiment 1 — Permuted MNIST (Start Here)

**RUN THIS FIRST**

This is the first experiment to run. It validates the full SOMA system on a standard continual learning benchmark. Runs in ~15 minutes on Kaggle T4. If this passes, the paper is real. If it doesn't, you need to debug before moving to GSM8K.

**Experiment 1: Permuted MNIST — Run Commands** *python*

```
# Full run command (from soma/ directory):
python -m soma.experiments.run_permuted_mnist

# Or with explicit arguments:
python -m soma.experiments.run_permuted_mnist \
--n_tasks 10 \
--n_train 1000 \
--n_test 200 \
--device cuda \
--seed 42

# Expected console output (one line per task):
# [Task 1/10]
# Action: SPAWN(cold) K: 0->1 Acc: 0.891 BT: 0.0000
# [Task 2/10]
# Action: SPAWN(cold) K: 1->2 Acc: 0.876 BT: 0.0000
# [Task 3/10]
# Action: SPAWN K: 2->3 Acc: 0.862 BT: -0.0210
# ...
# [Task 10/10]
# Action: UPDATE K: 7->7 Acc: 0.831 BT: -0.0380

# Final summary expected output:
# === SOMA Summary ===
# backward_transfer : -0.0380
# forward_transfer : 0.0220
# final_k : 7
# spawn_count : 6
# merge_count : 0
# tasks_completed : 10
# target_bt_met : True

# PASS criterion: backward_transfer > -0.05
# FAIL if backward_transfer <= -0.05 OR final_k >= 10
```

## 9.2 Running Baselines (Compare Against SOMA)

**Running Baseline Comparisons** *python*

```
# All baselines use the same data splits as SOMA.
# Run all three before comparing results.

# Baseline 1: Sequential (no protection)
python -m soma.experiments.baselines.sequential \
--dataset permuted_mnist --n_tasks 10
# Expected BT: around -0.18

# Baseline 2: EWC (Elastic Weight Consolidation)
python -m soma.experiments.baselines.ewc \
--dataset permuted_mnist --n_tasks 10 --lambda 1000
# Expected BT: around -0.09

# Baseline 3: Experience Replay
python -m soma.experiments.baselines.replay \
--dataset permuted_mnist --n_tasks 10 --memory_size 200
# Expected BT: around -0.07

# Compare all results:
python -m soma.utils.compare_results \
--runs soma ewc replay sequential
```

## 9.3 Running the Ablation (Critical for Paper)

**Running the Ablation Study** *python*

```
# The ablation removes each necessity signal one at a time.
# This is the core evidence that each of N1, N2, N3 contributes.

# Ablation A: SOMA with N2 and N3 only (remove N1)
python -m soma.experiments.run_permuted_mnist --disable_n1

# Ablation B: SOMA with N1 and N3 only (remove N2)
python -m soma.experiments.run_permuted_mnist --disable_n2

# Ablation C: SOMA with N1 and N2 only (remove N3)
python -m soma.experiments.run_permuted_mnist --disable_n3

# Ablation D: No RL (rule-based: if NECESSITY -> SPAWN, else UPDATE)
python -m soma.experiments.run_permuted_mnist --no_rl

# Expected results for paper Table 2:
# Full SOMA: BT -0.038 K=7
# No N1: BT -0.065 K=9 (over-spawns on lr-decay plateaus)
# No N2: BT -0.071 K=10 (doesn't detect subspace gaps)
# No N3: BT -0.058 K=8 (triggers on noise)
# No RL (rules): BT -0.047 K=8 (close but less efficient)
```

## 9.4 Experiment 2 — GSM8K Sequential (Stage 5)

**Experiment 2: GSM8K Sequential** *python*

```
# Run AFTER Experiment 1 passes. Requires Phi-3 Mini (3.8B).
# Download base model first (one-time, ~7GB on Kaggle):

from transformers import AutoModelForCausalLM, AutoTokenizer
model = AutoModelForCausalLM.from_pretrained(
'microsoft/Phi-3-mini-4k-instruct',
load_in_4bit=True, # 4-bit quantization: fits T4 VRAM
device_map='auto'
)
tokenizer = AutoTokenizer.from_pretrained('microsoft/Phi-3-mini-4k-instruct')

# Then run the experiment:
python -m soma.experiments.run_gsm8k_sequential \
--model_name microsoft/Phi-3-mini-4k-instruct \
--n_subtasks 3 \
--load_in_4bit True \
--device cuda

# Expected runtime: ~4 hours on Kaggle T4
# Expected PASS criterion: BT > -0.05 on 3-task GSM8K sequence
```

SECTION 10

# Unit Testing Plan

Run these tests BEFORE any experiment. Each test has a precise pass/fail criterion. A test that passes means you can trust that component when debugging experiments.

### `test_n1_plateau.py` — N1 — Loss Plateau

Create artificial loss sequence: decreasing for 100 steps, flat for 300+ steps.

Verify N1=False during decreasing phase, N1=True after 300 flat steps.

Create noisy loss (random jumps but trending down): verify N1=False throughout.

Verify plateau_score is continuous [0,1] and monotone during plateau.

**PASS: N1 correctly classifies 5/5 synthetic sequences.**

### `test_n2_subspace.py` — N2 — Subspace Saturation

Create basis P from unit vectors e_1, e_2, e_3.

Test grad = e_1: residual should be ~0 (in subspace). N2=False. PASS if residual < 0.05.

Test grad = e_7 (orthogonal to e_1..e_3): residual should be ~1. N2=True. PASS if residual > 0.95.

Test grad = 0.9*e_1 + 0.1*e_7: residual should be ~0.01. N2=False. PASS if residual < 0.05.

Test basis update: feed 50 gradients in direction e_7, call update_basis, verify e_7 now in basis.

**PASS: All 5 subspace tests pass with residual within 5% of expected value.**

### `test_n3_clustering.py` — N3 — Systematic Failure

Systematic set A: generate 50 gradients all ~parallel (small random perturbation).

Expected: N3=True, silhouette > 0.30, entropy < threshold.

Random set B: generate 50 gradients from uniform random directions.

Expected: N3=False, silhouette < 0.20.

Small set C: only 10 failures (below min_failures=25).

Expected: N3=False (insufficient data).

Calibration test: feed Set B as cold-start, verify threshold = 0.70 * Set B entropy.

**PASS: N3 correctly classifies all 4 synthetic test cases.**

### `test_grow_reward.py` — Reward + KL Gating

Test reward = 0 case: acc improves but old tasks forget equally. Verify reward near 0.

Test reward > 0 case: acc improves, old tasks unchanged, K constant. Verify reward > 0.

Test KL gating: create large update (KL > 0.10). Verify update is rejected.

Test KL rescale: same large update with lr*0.5. Verify KL drops below threshold.

**PASS: All 4 reward/gating tests produce expected sign and magnitude.**

### `test_router.py` — Router — No Forgetting

Register adapter 0 with embeddings from task 0.

Register adapter 1 with embeddings from task 1.

Route task 0 examples: verify routes to adapter 0 in >80% of cases.

Route task 1 examples: verify routes to adapter 1 in >80% of cases.

Add adapter 2 (task 2). Re-route task 0: verify still routes to adapter 0 (no forgetting).

**PASS: Routing accuracy > 80% for all adapters, including after new adapters added.**

`test_learn_integration.py` **Full 3-Task Smoke Test**

Run SOMA-LEARN on exactly 3 synthetic tasks.

Verify: cold start spawns 2 adapters after tasks 0 and 1.

Verify: task 2 triggers necessity check (does not always spawn).

Verify: TaskLog returned with correct field types and ranges.

Verify: BT is in [-1.0, 0.1] range after 3 tasks.

Verify: K is in [2, 4] range (between always-spawn and always-merge extremes).

**PASS: 3-task smoke test completes without errors, all fields in expected ranges.**

SECTION 11

# Experiment Design

## 11.1 Experiment 1 — Permuted MNIST

| Parameter | Value |
|---|---|
| **Purpose** | Validate SOMA on standard CL benchmark. Establish baseline comparison. |
| **Tasks** | 10 tasks. Each task = MNIST with a different fixed random pixel permutation applied. |
| **Data per task** | 1,000 training examples, 200 test examples. Sampled from original 60,000 MNIST images. |
| **Input** | 784-dimensional flattened pixel vector, normalised to [0,1]. |
| **Labels** | 10 classes (digits 0-9). Same classes across all tasks — only input distribution changes. |
| **Base model** | 3-layer MLP: Linear(784,256) -> ReLU -> Linear(256,128) -> ReLU -> Linear(128,10). Frozen. |
| **LoRA adapter** | Adapts the hidden layer: A is [8,128], B is [128,8]. rank=8. 2,048 trainable parameters. |
| **Training per task** | 200 gradient steps per task. Batch size 64. AdamW lr=1e-3. |
| **Why this first** | Runs in minutes. Standard benchmark with known baselines. Permutation = guaranteed systematic gap. |
| **Expected N3** | N3 should fire on ~80% of tasks. Permutations are maximally systematic transformations. |
| **Expected K** | K=7-9 after 10 tasks. SOMA should merge 1-2 pairs of similar permutations. |
| **PASS criterion** | BT > -0.05 AND K < 10. Both must be satisfied simultaneously. |
| **FAIL definition** | BT <= -0.05 OR K >= 10. Either failure means the system is broken. |

## 11.2 Experiment 2 — GSM8K Sequential Subtasks

| Parameter | Value |
|---|---|
| **Purpose** | Validate SOMA on real language tasks. Demonstrate transfer beyond vision benchmark. |
| **Base model** | microsoft/Phi-3-mini-4k-instruct (3.8B parameters). Load in 4-bit for T4 VRAM. |
| **Tasks** | 3 GSM8K subtasks: single-step arithmetic, multi-step reasoning, word problems. |
| **Data per task** | 500 training examples, 100 test examples from official GSM8K splits. |
| **LoRA adapter** | rank=8, target_modules=['q_proj','v_proj']. ~2M trainable parameters per adapter. |
| **Training per task** | 100 gradient steps (limited by Kaggle time quota). Batch size 4. lr=2e-4. |
| **Metric** | Accuracy = fraction of test examples where model produces correct final numerical answer. |
| **PASS criterion** | BT > -0.05 on the 3-task sequence. Replicate Experiment 1 result on language tasks. |
| **Expected runtime** | ~4 hours on Kaggle T4. Run overnight. |

SECTION 12

# Baselines

SOMA must be compared against these three methods on both experiments. Without these comparisons, the paper has no contribution claim.

### Sequential Fine-tuning

`experiments/baselines/sequential.py`

**Expected BT: -0.18**

The no-protection baseline. Fine-tune the same model sequentially on each task. No anti-forgetting mechanism. Establishes the worst-case ceiling. Implementation: just train in a loop, no freezing, no adapters. Run with same hyperparameters as SOMA training (lr=1e-3, 200 steps).

### EWC (Elastic Weight Consolidation)

`experiments/baselines/ewc.py`

**Expected BT: -0.09**

Kirkpatrick et al. 2017. After each task, compute the Fisher information matrix (diagonal approximation). Add a regularisation penalty to subsequent training: lambda * sum_i F_i * (theta_i - theta_i_*)^2. lambda=1000 is the standard value from the paper. Same model, same training loop, just adds the penalty term to the loss.

### Experience Replay

`experiments/baselines/replay.py`

**Expected BT: -0.07**

Store memory_size=200 examples from each past task. During training on new task, mix in memory_per_task=20 examples per old task. Simplest form: no generative model, just stored examples. This is the 'experience replay' baseline standard in CL literature. Same model as sequential. Extra memory: 200 * n_tasks examples.

SECTION 13

# Metrics Reference

Precise definitions of every metric used in Paper 1. These definitions must match the implementation exactly.

## 13.1 Backward Transfer (BT)

```
BT = (1/(T-1)) * sum_{i=1}^{T-1} [A(M_T, T_i) - A(M_i, T_i)]
```

*where:*

```
A(M_t, T_i) = accuracy on task T_i after training on task T_t
A(M_i, T_i) = accuracy on task T_i immediately after training on it (peak accuracy)
T = total number of tasks completed
```

BT = 0 means perfect retention. BT = -0.18 means severe forgetting. Paper 1 target: BT > -0.05.

## 13.2 Forward Transfer (FT)

```
FT = (1/(T-1)) * sum_{i=1}^{T} [A(M_{i-1}, T_i) - A(random, T_i)]
```

```
where A(M_{i-1}, T_i) = accuracy on T_i BEFORE seeing any T_i training data.
```

FT measures whether past learning helps on future tasks. A(random) = 0.10 for 10-class problems. Target: FT > +0.02 per month of training.

## 13.3 Per-Parameter Normalised Accuracy

```
PPA = accuracy / log(1 + n_params / n_params_baseline)
```

This normalises accuracy by parameter count so SOMA's growing architecture is fairly compared to fixed-size baselines. Used as secondary metric.

## 13.4 Necessity Signal Precision/Recall

```
Precision = TP / (TP + FP) where TP = growth that improved BT, FP = growth that harmed BT
```

```
Recall = TP / (TP + FN) where FN = tasks that needed growth but NECESSITY was False
```

Target precision > 0.75. Target recall > 0.70. Both measured over the full task sequence.

## 13.5 Override Rate

```
override_rate = n_forced_merges / n_tasks
```

Fraction of tasks where growth ceiling forced a MERGE action. If override_rate > 0.15, max_K is too low. Target: override_rate < 0.15.

SECTION 14

# Validation Criteria

Binary PASS/FAIL for each component and each experiment. Do not proceed to the next stage until the current stage passes.

| Component | What is measured | PASS criterion | FAIL definition |
|---|---|---|---|
| **N1 unit test** | N1=True on plateau, N1=False on noisy improving loss | **PASS: >95% correct** | **FAIL: any misclassification on clear cases** |
| **N2 unit test** | High residual on orthogonal gradient, low on same-subspace gradient | **PASS: within 5% of expected value** | **FAIL: residual direction wrong** |
| **N3 unit test** | N3=True on parallel failure grads, N3=False on random failure grads | **PASS: all 4 synthetic cases correct** | **FAIL: any synthetic case wrong** |
| **Router test** | Routes same-task inputs to correct adapter after adding new adapters | **PASS: >80% routing accuracy per adapter** | **FAIL: accuracy drops after adding adapter** |
| **Smoke test** | 3-task end-to-end run completes, BT in valid range | **PASS: runs without error, BT in [-1, 0.1]** | **FAIL: any exception, or BT out of range** |
| **Exp 1 — SOMA** | Permuted MNIST 10 tasks | **PASS: BT > -0.05 AND K < 10** | **FAIL: BT <= -0.05 OR K >= 10** |
| **Exp 1 — ablation** | Each signal degrades BT when removed | **PASS: removing Ni increases BT loss by >0.01** | **FAIL: removing a signal has no effect** |
| **Exp 2 — SOMA** | GSM8K 3-task sequence | **PASS: BT > -0.05** | **FAIL: BT <= -0.05** |
| **Paper comparison** | SOMA outperforms all three baselines on BT | **PASS: SOMA BT > EWC BT > Replay BT** | **FAIL: any baseline beats SOMA** |

SECTION 15

# Development Timeline

14 weeks from now to paper submission. Each stage has a single deliverable. Do not skip validation before moving to the next stage.

## 1

### Write and run N2 + N3 unit tests

*Weeks 1-2*

Create tests/test_n2_subspace.py with 5 synthetic gradient tests.

Create tests/test_n3_clustering.py with 4 synthetic failure set tests.

Run: python -m pytest tests/test_n2_subspace.py tests/test_n3_clustering.py -v

Fix any bugs in necessity.py until all tests pass.

No GPU needed. Runs on CPU in under 1 minute.

**PASS: All 9 tests pass (5 N2 + 4 N3)**

## 2

### Write N1 + router + reward unit tests

*Weeks 2-3*

Create tests/test_n1_plateau.py, tests/test_router.py, tests/test_grow_reward.py.

Run full test suite: python -m pytest tests/ -v

Fix any failures in necessity.py, grow.py, router.py.

Create tests/test_learn_integration.py smoke test (3 synthetic tasks).

**PASS: Full test suite passes: 18+ tests, 0 failures**

## 3

### Implement baselines and run Experiment 1 (rule-based SOMA)

*Weeks 3-5*

Build experiments/baselines/sequential.py, ewc.py, replay.py.

Replace RL policy in grow.py with fixed rules: NECESSITY=True->SPAWN, else UPDATE.

Run: python -m soma.experiments.run_permuted_mnist --no_rl

Run all three baselines. Record BT and K for each.

Expected: rule-based SOMA BT around -0.047, K around 8.

**PASS: Rule-based SOMA BT > -0.08 AND K < 10**

## 4

### Enable RL policy, run ablation, finalise Experiment 1

*Weeks 5-8*

Enable GrowthPolicy in grow.py (remove --no_rl flag).

Run: python -m soma.experiments.run_permuted_mnist (full SOMA with RL).

Run ablation: --disable_n1, --disable_n2, --disable_n3 variants.

Expected: RL-SOMA BT better than rule-based by at least 0.005.

Expected: each ablation variant has worse BT than full SOMA.

Generate Table 1 (comparison vs baselines) and Table 2 (ablation).

**PASS: Full SOMA BT > rule-based SOMA BT. All ablation variants worse than full SOMA.**

## 5

### Build and run Experiment 2 (GSM8K + Phi-3)

*Weeks 8-12*

Build experiments/run_gsm8k_sequential.py with Phi-3 Mini integration.

Download model: microsoft/Phi-3-mini-4k-instruct with 4-bit quantization.

Run: python -m soma.experiments.run_gsm8k_sequential (overnight on Kaggle T4).

Compare against EWC and replay baselines on GSM8K.

Generate learning curves (Figure 1) and comparison table (Table 3).

**PASS: SOMA GSM8K BT > -0.05 AND beats EWC and replay**

## 6

### Write Paper 1

*Weeks 12-14*

LaTeX structure: Abstract, Intro, Related Work, Method, Experiments, Ablation, Conclusion.

Target: EMNLP 2026 (February submission) or NeurIPS 2026 (May submission).

Release code on GitHub under Wakasa Labs before submission.

Abstract must lead with the BT number: 'SOMA achieves BT = -0.038...'

Register Wakasa Labs formally (unlocks grant eligibility for paper 2 funding).

**PASS: Paper submitted. Code on GitHub.**

SECTION 16

# Compute Budget

| Task | Platform | GPU | Est. time | Est. cost | Notes |
|---|---|---|---|---|---|
| Unit tests (all) | Local CPU | None | < 5 min | $0 | No GPU needed for unit tests |
| Exp 1: SOMA run | Kaggle free | T4 (15.8GB) | 15 min | $0 | Free 30h/week quota |
| Exp 1: 3 baselines | Kaggle free | T4 | 30 min total | $0 | 5-10 min each |
| Exp 1: ablation (4 variants) | Kaggle free | T4 | 1 hour total | $0 | 15 min each |
| Exp 2: Phi-3 download | Kaggle | T4 | 30 min DL | $0 | ~7GB model, one-time |
| Exp 2: SOMA GSM8K | Kaggle free | T4 | 4 hours | $0 | Run overnight |
| Exp 2: 2 baselines | Kaggle free | T4 | 3 hours total | $0 | EWC + replay |
| Reproducibility run | Colab Pro or Kaggle | A100 or T4 | 6 hours | ~$18 | For final paper submission |
| Contingency buffer | Colab Pro | A100 | ~10 hours | ~$115 | If bugs require extra runs |
| **TOTAL** | | | | **< $133** | **Well within Wakasa Labs target** |

SECTION 17

# What Is Still Open

Honest accounting of what is not yet solved. Paper 1 can proceed with partial solutions for Problems 2, 3, and 5. Problems 8 is deferred entirely to Paper 3.

**Problem 2 — Paper 1 (partial)**

### Merge Operation

Current implementation averages the two adapters: merged = (B1+B2)/2, (A1+A2)/2. This is known to be suboptimal — it dilutes both adapters. TIES-Merging (NeurIPS 2023) is a better candidate but was designed for full fine-tuning. Fisher-weighted merging is the correct approach but requires Fisher matrices. For Paper 1: the merge action rarely triggers in experiments (K stays below max_K). Use average merge as a placeholder and note it as a limitation in the paper.

**Problem 3 — Paper 1 (partial)**

### Router Forgetting

Current prototype router does not use gradient updates, so it cannot forget by construction. However, prototype quality may degrade as more adapters are added (routing confidence drops). For Paper 1: measure routing_confidence over time. If it stays > 0.70, report as solved. If it drops, acknowledge as a limitation and propose learned-prototype update as future work.

**Problem 5 — Paper 1 (eval only)**

### Fair Per-Parameter Metric

SOMA grows parameters over time (K adapters * adapter_size). A fair comparison with fixed-parameter baselines requires normalisation. Proposed metric: PPA = accuracy / log(1 + n_params / baseline_params). This is a secondary metric. Primary metric is BT. For Paper 1: report both BT and PPA. Reviewers will ask about parameter count.

**Problem 8 — Paper 3 only**

### Mutation Mapping

Level 3 of the SOMA architecture requires a learned mapping from failure categories to architectural mutations (rank increase, layer addition, etc.). This requires a meta-dataset of (failure_type, architecture_change, outcome) triples. Not needed for Paper 1 or 2. Deferred entirely to Paper 3.

SECTION 18

# Paper 1 Hypothesis

The falsifiable claim that Paper 1 exists to test. Everything in this document serves this one statement.

**PAPER 1 CENTRAL HYPOTHESIS**

HYPOTHESIS: A continual learning system using RL-guided selective capacity growth, triggered by a three-signal necessity detector (loss plateau + subspace saturation + systematic failure clustering), achieves Backward Transfer > -0.05 on sequential task learning benchmarks, using fewer than n_tasks adapters, outperforming EWC (BT ~-0.09), experience replay (BT ~-0.07), and sequential fine-tuning (BT ~-0.18) when measured per-parameter.

### The hypothesis is FALSIFIED if:

- BT <= -0.05 on either Permuted MNIST or GSM8K sequential.
- Target BT is only achieved with K >= n_tasks (one adapter per task) — no efficiency gain.
- Any baseline (EWC, replay, sequential) achieves better BT than SOMA.
- Removing any of N1, N2, N3 from the conjunction produces identical BT (signals are redundant).
- K grows without bound and requires manual intervention to stay bounded.

### The hypothesis is CONFIRMED if:

- BT > -0.05 on both benchmarks with K < n_tasks.
- Ablation shows each Ni contributes: removing any one increases forgetting by > 0.01.
- SOMA outperforms all three baselines on BT while using comparable or fewer parameters.
- Results are reproducible across 3 different random seeds.
- Runtime is feasible on Kaggle free tier (< 30 hours total GPU time).

**ENGINEERING NORTH STAR**

The BT number is the paper. Everything else — the algorithm, the architecture, the necessity signals, the RL policy — exists to produce a BT > -0.05. When you are in engineering mode, every decision should be traceable to: does this help BT? If you can't answer that, the decision is premature.